

Patrick Murray

Professor Rivas

Database Management

5 April 2026

Part A:

1: A transaction is a group of database actions that are grouped together as a single unit of instructions. Either all of the steps happen or none of them do, this ensures that there is no loss of data if something goes wrong in the middle of a series of changes to the instances.

2: A commit means the changes to the database done through the transaction are permanent.

3: A rollback means that the changes that the transaction did will not be made permanent and they will actually be reversed.

4: Atomicity means that the transaction happens all at once or it doesn't happen at all. Consistency ensures that the transactions should leave the database in a coherent state that is consistent with its laws. Isolation ensures that transactions should not interfere with another in a harmful manner. Durability ensures that once a transaction is committed, it stays done and the changes are concrete even if the system implodes right after.

Part B:

Query:

```
select * from Accounts;
```

```
BEGIN;
```

```
UPDATE Accounts
```

```
SET balance = balance -100
```

```
WHERE account_id = 1;
```

```
UPDATE Accounts
```

```
SET balance = balance + 100
```

```
WHERE account_id = 2;
```

```
COMMIT;
```

```
select * from Accounts;
```

Before: 3 "Cara" 200

1 "Ava" 500

2 "Ben" 300

After: 3 "Cara" 200

1 "Ava" 400

2 "Ben" 400

Explanation: This should be treated only as one unit of work/ as a transaction because if you broke this down into chunks like doing the transaction from account 1 first, you risk having the database lose information if something goes wrong during the individual steps. By having it as one unit of transactions, either all of it happens or none at all so you eliminate the possibility of losing important data if something goes wrong half way through the transaction.

Part C:

Query:

```
SELECT * FROM Accounts;
```

```
UPDATE Accounts
```

```
SET balance = balance +100
```

```
WHERE account_id = 1;
```

```
Rollback;
```

```
SELECT * FROM Accounts;
```

Before: 3 "Cara" 200

1 "Ava" 500

2 "Ben" 300

After: 3 "Cara" 200

1 "Ava" 500

2 "Ben" 300

Explanation: The rollback reset the changes that would have been made to the database by the transaction. Instead of Ava having 600 dollars, she still has 500 because the changes were reversed because of the rollback.

Part D:

1: This statement corresponds with A (Atomicity) because either all of the transfers happen or none of them do, ensuring no data is lost.

2: This statement corresponds with I (Isolation) because isolation ensures that none of the transactions interact with each other negatively. If two students were to register for a class at the same time, this rule would prevent unwanted side effects and ensure database stability.

3: This statement corresponds with D (Durability) because having the information still be there and saved after a crash is the function of durability whose purpose is to protect the integrity of the database and its information even if it crashes.

4: This statement corresponds with C (Consistency) because consistency makes sure that after a transaction happens the database still holds true to its rules and makes sense.

Part E:

1: If both updates were handled correctly, the final balance of the account would be 350.

2: If the updates were handled incorrectly, the final balance could be 450.

3: The property of ACID most at risk here is definitely isolation because these two transactions could happen at the exact same time, causing errors in the databases.

4: Locking would ensure these changes happen one at a time, not the same time. This would ensure no errors would be present in the database.

Part F:

1: A transaction is more than just running some SQL because it's a way to update data in a way that, if done right, eliminates the possibility of data getting corrupted or not correctly updated. It is also flexible with the added benefit of being able to rollback or commit changes.

2: A commit is important because it allows you to make sure the changes to the data are permanent.

3: A rollback is important because it allows you to reverse changes to your data that could be harmful if a certain condition arises.

4: A database will need isolation when a lot of users are active because if there isn't isolation, a lot of overlapping changes can happen which can lead to really bad data issues as seen with Part E. With isolation, you don't have to worry about that since the transactions are handled one at a time, separately.

Github: